On Wagner's k-Tree Algorithm Over Integers *

Haoxing Lin and Prashant Nalini Vasudevan

National University of Singapore
haoxingl@comp.nus.edu.sg, prashant@comp.nus.edu.sg

Abstract. The k-Tree algorithm [49] is a non-trivial algorithm for the average-case k-SUM problem that has found widespread use in cryptanalysis. Its input consists of k lists, each containing n integers from a range of size m. Wagner's original heuristic analysis [49] suggested that this algorithm succeeds with constant probability if $n \approx m^{1/(\log k+1)}$, and that in this case it runs in time $O(kn)$. Subsequent rigorous analysis of the algorithm [39, 47, 33] has shown that it succeeds with high probability if the input list sizes are significantly larger than this.
We present a broader rigorous analysis of the k-Tree algorithm, showing upper and lower bounds on its success probability and complexity for any size of the input lists. Our results confirm Wagner's heuristic conclusions, and also give meaningful bounds for a wide range of list sizes that are not covered by existing analyses. We present analytical bounds that are asymptotically tight, as well as an efficient algorithm that computes (provably correct) bounds for a wide range of concrete parameter settings. We also do the same for the k-Tree algorithm over $\mathbb{Z}_m$. Finally, we present extensive empirical evaluation of the k-Tree algorithm and demonstrate the tightness of our results.

1 Introduction

The (average-case) k-SUM problem is a basic computational problem that comes up often in cryptanalysis [46, 49, 21, 8, ...]. In this problem, given k lists of n integers each drawn uniformly at random from the range $\{-\lfloor m/2 \rfloor, \ldots, \lfloor m/2 \rfloor\}$, the task is to find a set of k integers, one from each list, such that their sum is 0. Such sets exist with high probability once the list size n is larger than $m^{1/k}$; and in this case there is a simple meet-in-the-middle algorithm for the problem that runs in time $\tilde{O}(m^{1/2})$. The only known algorithms that do significantly better than this are the k-Tree algorithm [49], and its extensions [40, 41, 19] and variants [10, 39, 33].

The k-Tree algorithm, as formulated by Wagner [49], is described in Figure 1.[1] Wagner provided heuristic arguments indicating that this algorithm succeeds with constant probability if the input lists are of size[2] $n \approx m^{1/(\log k+1)}$, in which

* The full version of this paper is available at https://eprint.iacr.org/2024/1612.
[1] See the algorithm figure in Analysis section in the paper's full version for a more detailed presentation.
[2] This list size arises naturally from a first-moment-based analysis of the algorithm, which is presented as part of our technical overview in Section 3.

The k-Tree algorithm

Parameters: $k, n, m \in \mathbb{N}$, with k being a power of 2

Input: Lists $L_1, \ldots, L_k$, each consisting of n integers from $\{-\lfloor m/2 \rfloor, \ldots, \lfloor m/2 \rfloor\}$

Output: Indices $\ell_1, \ldots, \ell_k \in [n]$, or a failure symbol $\perp$

Procedure:

1. Initialization:
   - Set $p = m^{-1/(\log k+1)}$
   - For each $i \in [k]$, denote the list L_i by L_i^0
   - Set $\tau \leftarrow m/2$
2. For d from 1 to $\log k$:
   - Set $\tau \leftarrow p \cdot \tau$
   - For $i \in \left[\frac{k}{2^d}\right]$:
     compute $L_i^d \leftarrow \left\{(a+b) \;\middle|\; a \in L_{2i-1}^{d-1} \wedge b \in L_{2i}^{d-1} \wedge |a+b| \leq \tau\right\}$
3. If $L_1^{\log k}$ contains 0, output the indices in the input lists that led to this sum. Otherwise output $\perp$.

Fig. 1: The k-Tree algorithm over Integers

case it runs in time $O(k \cdot m^{1/(\log k+1)})$. Even for modest values of k (say 4 or 8), this runtime is significantly better than the earlier $\tilde{O}(m^{1/2})$ for large values of m. For this reason, the k-Tree algorithm has been used repeatedly in the cryptanalysis of signatures [49, 21, 8], hash functions [17, 13], identification schemes [37], symmetric-key encryption schemes [32], etc., to provide simple attacks that perform significantly better than brute-force.

While the k-Tree approach can be used to solve the k-SUM problem over other groups as well, we focus on its application to k-SUM over integers and modular integers (that is, $\mathbb{Z}_m$), as these have been particularly significant in cryptography. A prominent example is in solving the ROS problem [23], which underlies the security of several discrete logarithm-based blind signatures including Schnorr [46] and Okamoto-Schnorr [44] signatures. Wagner [49] demonstrated that the k-Tree algorithm over $\mathbb{Z}_m$ provides an efficient solver for ROS [3], leading to forgery attacks on these signature schemes. More recently, Benhamouda et al. [8] achieved a breakthrough by showing that the ROS problem arising out of these schemes is solvable in polynomial time when the number of concurrent signing sessions exceeds $\log m$. They further developed a generalized attack that provides flexible trade-offs between the number of sessions and attack complexity by combining the k-Tree algorithm with their polynomial-time techniques.

[3] Assuming k-Tree succeeds with constant probability, see Conjecture 1 in their paper.

In spite of such widespread relevance, our understanding of the behavior of the k-Tree algorithm is somewhat limited. There is empirical evidence that Wagner's aforementioned heuristic conclusion regarding the case of list sizes $\approx m^{1/(\log k+1)}$ is valid.[4] But his analysis was based on assumptions that are too coarse-grained to lead to meaningful conclusions for other list sizes. Rigorous analysis of the algorithm has since been carried out by Lyubashevsky [39], Shallue [47], and Joux et al. [33]. These provide good bounds on the behavior of the algorithm when the list sizes are significantly larger, but have not been able to confirm Wagner's conclusion (see Section 4 for details). These leave significant gaps in our understanding. For instance, we still do not have the means to provide good answers to basic questions like the following, even heuristically:

1. What is the probability that the algorithm will succeed if the lists are of size $(10 \cdot m^{1/(\log k+1)})$ or $(0.1 \cdot m^{1/(\log k+1)})$?
2. What size of lists is necessary or sufficient for the algorithm to succeed with probability 0.01 or $1/\log m$?
3. What is the complexity of the algorithm for these list sizes?

Answers to these questions for some pairs of values (m, k) for which these list sizes are not too large – e.g., $(2^{64}, 4)$, $(2^{256}, 128)$ – can be obtained empirically, but it is not clear how to extrapolate these to other settings that may come out of concrete parameter choices in constructions – e.g., $(2^{256}, 4)$. Our objective in this work is to provide a tight analysis of the k-Tree algorithm that can help researchers answer questions like the ones above and thus understand more accurately the power of attacks that make use of the k-Tree algorithm.

Paper Outline. We summarize our results in Section 2, and provide an overview of our techniques in Section 3. In Section 4, we describe prior work on this topic, present comparisons to our results, and discuss other related work. In the full version's Analysis section, we provide the complete proof of our main theorem regarding the behavior of the k-Tree algorithm. In Section 5, we present sample computations of our bounds for concrete parameters; and in Section 6, we present experimental evaluations of their tightness.

Due to page limits, we defer the complete rigorous proofs of our theorems to the full version's Analysis section, noting that the overview in Section 3 goes through most of their essential components. Presentation of the pseudo-code for computing tighter bounds on the behavior of the algorithm is also similarly deferred to the full version (Computed Bounds section). We also defer our analysis of the k-Tree algorithm over $\mathbb{Z}_m$ to Analysis' $\mathbb{Z}_m$ subsection in the paper's full version, noting that the bounds in this case are very close to those in the case of integers presented here.

[4] See, for instance, the results of our experiments presented in Section 6.

2 Results

Recall that we are interested in the performance of the k-Tree algorithm when given as input k lists of n numbers each drawn uniformly at random from $\{-\lfloor m/2 \rfloor, \ldots, \lfloor m/2 \rfloor\}$. Here k is a power of 2. With these parameters, we define the *complexity* of the k-Tree algorithm to be the expected total size of all the lists that are generated in the course of its execution.[5]

Analytical Bounds. Our first set of results are analytical bounds on the success probability and complexity of the k-Tree algorithm for any list size.

Theorem 1. *Consider any $k, n, m \in \mathbb{N}$, where $k \geq 4$ is a power of 2 and $m > 30^{\log k+1}$. Set $p = m^{\frac{-1}{\log k+1}}$ and $c = n/m^{\frac{1}{\log k+1}}$. The probability of success of the k-Tree algorithm is bounded as:*

$$\frac{c^k}{1 + c^k \cdot \left(1 + \frac{k}{n}\right)^k} \cdot (1 - 150p)^k \leq \Pr\left[k\text{-Tree succeeds}\right] \leq c^k \cdot (1 + 37p)^k$$

Its complexity is bounded as follows:

$$\text{Complexity of } k\text{-Tree} \in kn \cdot \left(1 + \sum_{d \in [\log k]} \frac{c^{2^d - 1}}{2^d}\right) \cdot (1 \pm 37p)^{k-1}$$

Asymptotically, the following simpler bound can be inferred from the above theorem if k is not too large in relation to the other parameters.

Corollary 1. *Consider the same parameters as in Theorem 1, and in addition suppose that $k = o(1/p)$ and $k = o(n^{1/2})$. Then we have:*

$$\frac{c^k}{1 + c^k} \cdot (1 - o(1)) \leq \Pr\left[k\text{-Tree succeeds}\right] \leq c^k \cdot (1 + o(1))$$

The setting $c = 1$ corresponds to the choice of $n = m^{1/(\log k+1)}$ for the lists sizes suggested by Wagner's heuristic argument [49]. As long as k is not too large, the above results confirm that in this case the k-Tree algorithm succeeds with some probability roughly between $1/2$ and 1. In addition, they also allow one to compute list sizes and the resulting complexity that is sufficient for the algorithm to succeed with probability, say, 0.01; or even the complexity that is *necessary* for the algorithm to succeed with probability at least 0.01. This enables answering the various kinds of questions about the algorithm presented earlier.

Computed Bounds. For certain concrete parameter values (m, k) (for example, $(2^{256}, 1024)$), Theorem 1 provides bounds that, while provably correct and

[5] See Remark 3 at the end of this subsection for a brief discussion of this complexity measure.

tight in the asymptotic regime, might be too loose to be practically useful. This looseness stems from two sources: the inherent limitations of our proof technique, and the approximations made to simplify intermediate expressions in our analytical derivation. The second source – the analytical approximations – turn out to contribute more significantly to the gap between the upper and lower bounds in the theorem statement above.

We eliminate this second source of inaccuracy by implementing our entire proof as a program that, instead of attempting to produce simple analytical expressions, explicitly computes the resulting probability and complexity bounds given values for the parameters k, m, and n. The pseudo-code for the relevant algorithms are presented in the full version's Computed Bounds section, and our implementation of it is available in the associated repository [38].

Theorem 2. *Consider any $k, n, m \in \mathbb{N}$, where $k \geq 4$ is a power of 2, and let $p = m^{\frac{-1}{\log k + 1}}$. There is an algorithm that outputs the upper and lower bounds on the success probability of the k-Tree algorithm, and there is also an algorithm that outputs the upper and lower bounds of its complexity. Further, these algorithms run in time* $\mathrm{polylog}(k, m, n)$.

We find that this tightening does significantly strengthen the resulting bounds for various concrete values of (m, k). We plot the bounds on success probability from both these theorems for a couple of pairs of values of (m, k) in Figure 2 for illustration, and present more examples of these bounds and their corollaries in Section 5. For the rest of this section, we solely use the bounds from Theorem 2 as they are more accurate.

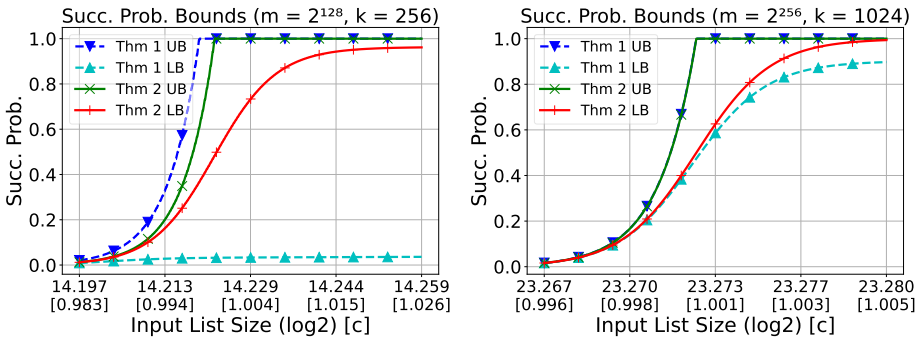


Fig. 2: Plot of upper and lower bounds on success probabilities against the input list size n. Bounds from both Theorems 1 and 2 are represented. The quantity c is defined to be the ratio $n/m^{1/(\log k+1)}$. Labels on the x-axis are $\log_2(n)$, with the corresponding value of c in square brackets.

Using Our Bounds. We observe that our results are useful in a few different

ways. In applications of the k-Tree algorithm where the size of the range m and the number of lists k are fixed and not in the control of the algorithm designer (e.g. the attack in [17], solving the ROS problem [49, 8]), our bounds can be used to determine the list size (and the corresponding complexity) that is sufficient to achieve a specific desired probability of success. This may be done analytically using Theorem 1 if k is relatively small, or computationally using Theorem 2 (with a binary search for n) for larger values of k.

In applications where the range m is fixed but k may be determined by the algorithm designer (e.g. attacks in [21, 32], the attacks on hash functions in [49]), our bounds can also be used to select the best value of k together with the best value of n (as the complexity depends on both) for a desired probability of success. In Figure 3, we present some sample computations of this form, covering some of the parameter settings from the papers cited above as examples.

Finally, our upper bounds on the probability of success of the k-Tree algorithm can be used to obtain concrete lower bounds on the complexity of k-Tree-based attacks on candidate cryptographic constructions that achieve any given probability of success.

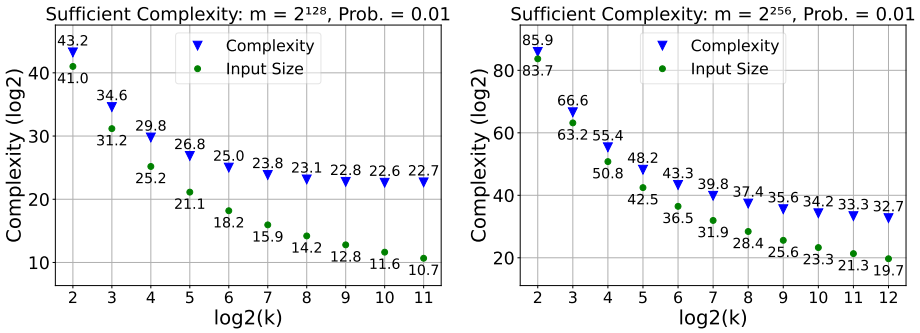


Fig. 3: Plots of the complexity of the k-Tree algorithm that is sufficient to provably achieve success probability of 0.01, against k. Input list size in each case is chosen so that the lower bound on success probability is slightly larger than 0.01. Computed using Theorem 2.

Experimental Evaluation. To understand the tightness of our upper and lower bounds, we empirically estimate (by running the k-Tree algorithm) the values of the quantities bounded in the theorems above for various values of the parameters k, m, and n, and compare them to our predictions. We refer the reader to Section 6 for details of these experiments, and a summary of their results and implications.

Remark 1 (Variations in the Input Distribution). Throughout the paper, we only consider input lists where each element is sampled independently and uni-

formly at random from the range $[-m/2, m/2]$ (or $\mathbb{Z}_m$). In applications of the k-Tree algorithm, these conditions might not all be satisfied. In applying it to the Subset Sum problem (as in [39] or [30]), for instance, the lists that are constructed do not consist of fully independent numbers. In some of these cases, the k-Tree algorithm might work quite well regardless.

While Theorem 1 as written does not capture all the numerous possible variations in the conditions on the input lists for which the algorithm can be proven to work, the approach we take in our proof continues to work as long as: (i) each list is sampled independently of other lists, (ii) elements within each list are pairwise-independent, and (iii) each element in the lists is sufficently well-distributed over the range.

The last condition may be realized in various ways – being sufficiently close to uniform in statistical distance, being distributed over a large enough unstructured set that is far from being additively closed, etc.. Further, the other two conditions need not be met perfectly – being sufficiently close to being independent or pairwise-independent is sufficient. In particular, these conditions are satisfied by the lists constructed in the aforementioned application to the Subset Sum problem. In this manner, our techniques remain meaningful beyond the case of fully uniformly random inputs.

Remark 2 (Simple Optimizations). There are some simple optimizations to the k-Tree algorithm that can significantly improve its behavior in some parameter regimes. For instance, removing duplicates when computing the intermediate lists L_i^d ensures that the complexity of the algorithm is at most $O(k(n+m))$, which is smaller than the bounds produced by Theorems 1 and 2 for very large values of c.

For large values of c, the lower bound on the success probability produced by these theorems is roughly $(1 - c^{-k})$. But for such c, simply breaking up the lists into $\lfloor c \rfloor$ smaller lists with $m^{1/(\log k+1)}$ elements each and running k-Tree on each separately (effectively with "c"$= 1$) results in an amplified success probability of $(1 - e^{-\Omega(c)})$.

Remark 3 (Measure of Complexity). Depending on various incidental factors such as the hardware being used, its word-size in relation to m, etc., the actual running time of the optimal implementation of the k-Tree algorithm might be anything from a constant factor to a logarithmic factor larger than the total list size. However, in all cases, the total list size is the central quantity that determines the running time. So for the sake of generality and simplicity, we directly use this as our measure of complexity. This is the measure that has been used in previous analyses of the algorithm as well [47, 33]. Corresponding to the specific implementation of the algorithm being analyzed and the hardware used, etc., the predicted running times of the algorithm can be computed from this measure.

Remark 4 (Integers vs. $\mathbb{Z}_m$). So far, we have been discussing the k-SUM problem where the addition is over integers. In reality, the problem that is most often

useful in cryptanalysis, etc., is the problem where addition is done modulo m (that is, over $\mathbb{Z}_m$), often for some prime number m. In this case, the k-Tree algorithm is modified to perform all its additions modulo m instead of over integers, with $\mathbb{Z}_m$ being identified with the set $\{-\lfloor m/2 \rfloor, \ldots, \lfloor m/2 \rfloor\}$ in the natural manner. Prior analyses by Shallue [47] and Joux et al. [33] were also for k-Tree over $\mathbb{Z}_m$. However, as demonstrated in Analysis' $\mathbb{Z}_m$ subsection in the full version, the difference in the behavior of the k-Tree algorithm in these two cases is so small as to be beneath the precision with which we state our results in this section. So for simplicity, we continue to ignore this distinction in this section, though the rigorous proofs in the full version's Analysis sectionwill take it into account.

Remark 5 (Parameter Choices). In the k-Tree algorithm as described in Figure 1 (as in Wagner's original description), we set $p = m^{-1/(\log k+1)}$ uniformly across all levels as it (1) ensures we expect at least one solution at the final level, and (2) achieves balanced expected list sizes at each level. Using different p would cause some intermediate lists to grow exponentially larger than the others in expectation. Minimizing the maximum list size therefore minimizes the overall complexity of the algorithm, making this choice of using the same p across all levels the optimal one.

Table 1: Table of Notations

Symbol	Description
Primary Parameters	
k	The number of input lists; assumed to be a power of 2.
n	The size (number of elements) of each input list.
m	The parameter defining the range of input integers.
p	The filtering parameter in the k-Tree algorithm, typically $m^{-1/(\log k+1)}$.
c	The normalized list size, defined as $c = n \cdot p$.
Sets, Lists, and Groups	
L_i	The i-th input list, for $i \in [k]$.
L_i^d	An intermediate list generated at level d of the k-Tree algorithm.
$[m]$	The set of integers $\{1, 2, \ldots, m\}$.
$\langle m \rangle$	The set of integers $\{-\lfloor m/2 \rfloor, \ldots, \lfloor m/2 \rfloor\}$.
$\mathbb{Z}_m$	The group of integers modulo m.
$\{0,1\}$	The set of binary values $\{0, 1\}$.
Algorithm and Analysis Variables	
τ	The threshold for the absolute value of sums in the 'Merge' subroutine.
$\bar{\ell}$	A tuple of indices $(\ell_1, \ldots, \ell_k)$, where $\ell_i \in [n]$.

Continued on next page

Table 1 – continued from previous page

Symbol	Description
$\delta(\bar{\ell}, \bar{\ell}')$	A binary vector in $\{0,1\}^k$ where the i-th bit is 0 if $\ell_i = \ell'_i$ and 1 otherwise.
$wt(\mathbf{b})$	The Hamming weight of a binary vector $\mathbf{b}$.
U_s	The uniform probability distribution over the set $\langle s \rangle$.
$\Delta_{\mathrm{MR}}(D_1, D_2)$	The Max-Ratio (MR) distance between two probability distributions D_1 and D_2.
Random Variables and Operators	
C	The random variable counting the number of solutions (occurrences of 0) in the final list $L_1^{\log k}$.
$C_{\bar{\ell}}$	An indicator random variable that is 1 if the elements indexed by $\bar{\ell}$ form a valid solution.
$\Lambda_{k,n,m,p}$	The random variable representing the total size of all lists generated by the algorithm.
$\mathbf{E}[X]$	The expected value of a random variable X.
$\Pr[E]$	The probability of an event E.
$T_{\mu,n,\nu}(k)$	A recursively defined function used for bounding the second moment of C.

3 Technical Overview

In this section, we present an overview of our proof of the bounds on the success probability of the k-Tree algorithm presented in Theorem 1. Our bounds on the complexity of the algorithm follow from similar arguments. The algorithms captured in Theorem 2 follow the same high-level approach as this proof, but involve some careful re-framing of its terms in order to enable efficient computation of some intermediate values. The entire rigorous proof is presented in the full version's Analysis section.

Our approach to these bounds is as follows: we define a random variable corresponding to the number of occurrences of "0" in the final list computed by the k-Tree algorithm, compute bounds on its first and second moments, and then use second-moment-based concentration bounds to bound the probability that this variable has non-zero value. Computing these moments, especially to levels of precision that provide meaningful bounds for concrete parameter settings, turns out to be challenging as this variable is quite complex. Among other things, we end up needing to use very specific measures of distance between probability distributions tailored to our needs, and carefully set up induction arguments over binary trees.

Setup. Recall that the input to the k-Tree algorithm is k lists of n integers, with each integer being drawn uniformly at random from the set $\{-\lfloor m/2 \rfloor, \ldots, \lfloor m/2 \rfloor\}$. Given these parameters, define the symbols $p = m^{-1/(\log k+1)}$, which is the filtering parameter in the algorithm, and $c = n \cdot p$, which represents the size of

the lists relative to the "default value" of $m^{1/(\log k+1)}$ suggested by heuristic arguments. For any non-negative integer s, we will denote by $\langle s \rangle$ such a set $\{-\lfloor s/2 \rfloor, \ldots, \lfloor s/2 \rfloor\}$.

For simplicity, we will fix the number of lists k to 4 in this overview (and so $p = m^{-1/3}$). In this case, the k-Tree algorithm is given as inputs $k = 4$ lists $L_1, \ldots, L_4$, each of size n, and proceeds as follows:

1. Compute the following lists:
   - $L_1^1 \leftarrow \left\{(a+b) \;\middle|\; a \in L_1 \wedge b \in L_2 \wedge (a+b) \in \langle mp \rangle\right\}$
   - $L_2^1 \leftarrow \left\{(a+b) \;\middle|\; a \in L_3 \wedge b \in L_4 \wedge (a+b) \in \langle mp \rangle\right\}$
   - $L_1^2 \leftarrow \left\{(a+b) \;\middle|\; a \in L_1^1 \wedge b \in L_2^1 \wedge (a+b) \in \langle mp^2 \rangle\right\}$
2. Succeed if L_1^2 contains a 0, fail otherwise.

Above, take the lists L_i^d to each be a multi-set instead of a set – for instance, if a specific value of $(a+b) \in \langle mp \rangle$ occurs twice as the sum of elements in L_1 and L_2, then two such values will be present in L_1^1. This makes the algorithm sub-optimal in terms of efficiency, but does not change its success probability and makes it easier to analyze. Define the following random variable as described earlier:

$$C = \text{number of occurrences of 0 in the list } L_1^2$$

The algorithm succeeds exactly when the value of this non-negative variable C is at least 1. So the probability of this latter event is what we will be concerned with in our analysis.

For any set of input lists $\{L_i\}$, the value of C may be computed by iterating through every possible value of the indices $\ell_1, \ldots, \ell_4 \in [n]$, and considering whether the tuple $(L_1[\ell_1], \ldots, L_4[\ell_4])$ passes all the "filters" of the algorithm and also sums to 0. For each such $\bar{\ell} = (\ell_1, \ldots, \ell_4)$, we capture this by defining a variable $C_{\bar{\ell}}$ that is 1 if all of the following events occur and is 0 otherwise:

$$\begin{aligned}
E_1 &\equiv (L_1[\ell_1] + L_2[\ell_2] \in \langle mp \rangle) \\
E_2 &\equiv (L_3[\ell_3] + L_4[\ell_4] \in \langle mp \rangle) \\
E_3 &\equiv (L_1[\ell_1] + L_2[\ell_2] + L_3[\ell_3] + L_4[\ell_4] \in \langle mp^2 \rangle) \\
E_4 &\equiv (L_1[\ell_1] + L_2[\ell_2] + L_3[\ell_3] + L_4[\ell_4] = 0)
\end{aligned}$$

Note that $C_{\bar{\ell}}$ is a random variable, with randomness coming from the numbers in the input lists. The variable C can now be written as follows:

$$C = \sum_{\bar{\ell} \in [n]^4} C_{\bar{\ell}} \tag{1}$$

We will be interested in the moments of this random variable C, which are functions of m and n (since we have already fixed k).

Heuristic Analysis. Wagner's heuristic analysis [49] of the k-Tree algorithm essentially relies on the following three assumptions (for any large enough integer s):

(a) When $\mathbf{E}[C] = 1$, the algorithm succeeds with constant probability.
(b) When x and y are sampled uniformly at random from $\langle s \rangle$, their sum $(x + y)$ is contained in $\langle sp \rangle$ with probability p.
(c) With x and y sampled uniformly at random from $\langle s \rangle$, the distribution of $(x + y)$, when conditioned on being contained in $\langle sp \rangle$, is uniform over $\langle sp \rangle$

Fix some tuple of indices $\bar{\ell} = (\ell_1, \ldots, \ell_4)$. Under assumptions (b) and (c), the expectation of $C_{\bar{\ell}}$ can be computed by computing the probability of events $E_1, \ldots, E_4$ as follows:

- Assumption (b) implies that events E_1 and E_2 each happens with probability p.
- Conditioned on these happening, assumption (c) implies that the sums $(L_1[\ell_1] + L_2[\ell_2])$ and $(L_3[\ell_3] + L_4[\ell_4])$ are uniformly distributed over $\langle mp \rangle$.
- So by (b), E_3 also happens with probability p.
- Finally, appealing to (c) again, conditioned on the first three events happening, the entire sum is distributed uniformly over $\langle mp^2 \rangle$, and so E_4 happens with probability $(mp^2)^{-1}$.

Thus, we have:

$$\begin{aligned}\mathbf{E}[C_{\bar{\ell}}] &= \Pr[E_1 \wedge \cdots \wedge E_4] \\ &= \Pr[E_1] \cdot \Pr[E_2] \cdot \Pr[E_3 \mid E_1 \wedge E_2] \cdot \Pr[E_4 \mid E_1 \wedge E_2 \wedge E_3] \\ &= p \cdot p \cdot p \cdot \frac{1}{mp^2} = \frac{p}{m} = p^4 \end{aligned} \tag{2}$$

The expected value of C is then:

$$\mathbf{E}[C] = \sum_{\bar{\ell} \in [n]^4} \mathbf{E}[C_{\bar{\ell}}] = n^4 \cdot p^4 = c^4 \tag{3}$$

Thus, when $c = 1$ (that is, $n = m^{1/3}$), this expectation is 1, and by assumption (a), the algorithm works with constant probability. This was the conclusion in [49]. In providing a rigorous analysis of the k-Tree algorithm, our objective is to remove the above assumptions. We show that assumptions (b) and (c), while not strictly true, are close enough to being true. Further, we show that a generalization of assumption (a) is true, which allows us to obtain bounds on the success probability of the algorithm for a wide range of values of c rather than just $c = 1$.

Relaxing the Assumptions. To compute the actual expectation of $C_{\bar{\ell}}$, we start by first showing that slightly relaxed versions of assumptions (b) and (c) above are true. Starting with assumption (b), it can be shown using some elementary computation (see Additional Preliminaries subsection in the full version) that:

$$\Pr_{x,y \leftarrow \langle s \rangle}[x + y \in \langle sp \rangle] \in \left(p - \frac{p^2}{4}\right) \pm O\left(\frac{1}{s}\right) \tag{4}$$

The right-hand side above is not exactly p, but for typical values of p and s, is quite close to it.

For assumption (c), we show that while the distribution of $(x+y)$ as described there is not actually uniform, it is close to the uniform distribution. The specific notion of distance we use, which we refer to as the *Max-Ratio (MR) distance* from Uniform, is defined as follows[6] for a distribution D over a domain S:

$$\Delta_{\mathrm{MR}}(U, D) = \frac{\max_{x \in S} D(x)}{\min_{x \in S} D(x)}$$

where $D(x)$ is the probability mass placed on x by the distribution D. This distance is at least 1 (which is achieved if D is the uniform distribution over S), and is potentially unbounded.

This notion of distance is particularly beneficial for our analysis for a couple of reasons. First, for any event E over the domain S, we can bound the probability of E happening under D in terms of the probability of E happening under the uniform distribution over S as follows:

$$\Delta_{\mathrm{MR}}(U, D)^{-1} \cdot \Pr_{x \leftarrow U}[E(x)] \leq \Pr_{x \leftarrow D}[E(x)] \leq \Delta_{\mathrm{MR}}(U, D) \cdot \Pr_{x \leftarrow U}[E(x)] \tag{5}$$

Second, this distance is easy to compute for distributions defined under conditioning, which can otherwise be difficult to reason about. In particular, suppose D is the distribution of $(x + y)$ conditioned on being in $\langle sp \rangle$ when x and y are uniformly drawn from $\langle s \rangle$. Then, this distance is as follows:

$$\begin{aligned}
\Delta_{\mathrm{MR}}(U, D) &= \frac{\max_{z \in \langle sp \rangle} \Pr_{x,y \leftarrow \langle s \rangle}[x + y = z \mid x + y \in \langle sp \rangle]}{\min_{z \in \langle sp \rangle} \Pr_{x,y \leftarrow \langle s \rangle}[x + y = z \mid x + y \in \langle sp \rangle]} \\
&= \frac{\max_{z \in \langle sp \rangle} \Pr_{x,y \leftarrow \langle s \rangle}[x + y = z] \cdot \Pr[x + y \in \langle sp \rangle]^{-1}}{\min_{z \in \langle sp \rangle} \Pr_{x,y \leftarrow \langle s \rangle}[x + y = z] \cdot \Pr[x + y \in \langle sp \rangle]^{-1}} \\
&= \frac{\max_{z \in \langle sp \rangle} \Pr_{x,y \leftarrow \langle s \rangle}[x + y = z]}{\min_{z \in \langle sp \rangle} \Pr_{x,y \leftarrow \langle s \rangle}[x + y = z]} \\
&\leq (1 + p) + O\left(\frac{1}{s}\right)
\end{aligned} \tag{6}$$

where the second equality follows from the Bayes theorem, and the inequality again follows from elementary computations. Again, for typical values of p and s, the right-hand side above is quite close to 1, indicating that the distribution is close to uniform. Putting together (5) and (6), with some further computation, we get the following effective relaxation of assumption (c). For any event $E(z)$ defined over domain $\langle sp \rangle$, we have:

$$\Pr_{x,y \leftarrow \langle s \rangle}[E(x + y) \mid (x + y) \in \langle sp \rangle] \in \Pr_{z \leftarrow \langle sp \rangle}[E(z)] \cdot \left[1 \pm \left(p + O\left(\frac{1}{s}\right)\right)\right] \tag{7}$$

[6] This is again a simplification. For the actual definition, see the formal one in the full section's Additional Preliminaries subsection.

For simplicity, in the rest of this overview we will ignore the $O(1/s)$ parts in the expressions (4) and (7) above. It is not a crucial part of the big picture, and in most reasonable parameter settings is much smaller than p anyway.

Computing the Expectation. With (4) and (7) in hand, we can now compute the expectation of $C_{\bar{\ell}}$. We essentially follow the earlier heuristic argument step-by-step, replacing the assumptions there with their true but relaxed versions.

Denote by $\overline{x}$ the vector $(x_1, \ldots, x_4)$, where $x_i \in \langle m \rangle$ will later be identified with $L_i[\ell_i]$; the variables x_1^1, x_2^1, and x_1^2 below will similarly initially be identified with $(x_1 + x_2)$, $(x_3 + x_4)$, and $(x_1^1 + x_2^1)$, respectively. We re-state the events in definition of $C_{\bar{\ell}}$ to be parameterized as follows for ease of manipulation:

$$\begin{aligned} E_1(x_1, x_2) &\equiv (x_1 + x_2 \in \langle mp \rangle) \\ E_2(x_3, x_4) &\equiv (x_3 + x_4 \in \langle mp \rangle) \\ E_3(x_1^1, x_2^1) &\equiv (x_1^1 + x_2^1 \in \langle mp^2 \rangle) \\ E_4(x_1^2) &\equiv (x_1^2 = 0) \end{aligned}$$

Along the lines of (2), the expectation of $C_{\bar{\ell}}$ for any $\bar{\ell}$ can be written as follows:

$$\begin{aligned} \mathbf{E}\left[C_{\bar{\ell}}\right] &= \Pr_{x_1,\ldots,x_4 \leftarrow \langle m \rangle} \left[\begin{array}{c} E_1(x_1, x_2) \wedge E_2(x_3, x_4) \wedge \\ E_3(x_1 + x_2, x_3 + x_4) \wedge E_4(x_1 + \cdots + x_4) \end{array} \right] \\ &= \Pr_{x_1,x_2 \leftarrow \langle m \rangle} \left[E_1(x_1, x_2)\right] \cdot \Pr_{x_3,x_4 \leftarrow \langle m \rangle} \left[E_2(x_3, x_4)\right] \\ &\quad \cdot \Pr_{x_1,\ldots,x_4 \leftarrow \langle m \rangle} \left[\begin{array}{c|c} E_3(x_1 + x_2, x_3 + x_4) \wedge E_4(x_1 + \cdots + x_4) & \\ & E_1(x_1, x_2) \wedge E_2(x_3, x_4) \end{array} \right] \end{aligned} \tag{8}$$

The first two terms in the product above can be bounded using (4) as follows:

$$\Pr_{x_1,x_2 \leftarrow \langle m \rangle} \left[E_1(x_1, x_2)\right] = \Pr_{x_3,x_4 \leftarrow \langle m \rangle} \left[E_2(x_3, x_4)\right] \approx \left(p - \frac{p^2}{4}\right) \tag{9}$$

The last term is more complex, but a useful observation here is that the events in the probability expression there only depend on the sums $(x_1 + x_2)$ and $(x_3 + x_4)$ rather than on the x_i's directly. Further, these events are conditioned on these sums being contained in $\langle mp \rangle$ (that is, conditioned on the events E_1 and E_2). If assumption (c) had been true, it would have implied that these sums are uniformly distributed over $\langle mp \rangle$, simplifying the probability expression considerably. We can do the same using (7), albeit with some loss as follows:

$$\begin{aligned} &\Pr_{x_1,x_2,x_3,x_4 \leftarrow \langle m \rangle} \left[\begin{array}{c|c} E_3(x_1 + x_2, x_3 + x_4) \wedge E_4((x_1 + x_2) + (x_3 + x_4)) & \\ & E_1(x_1, x_2) \wedge E_2(x_3, x_4) \end{array} \right] \\ &\in \Pr_{x_1^1,x_1^2 \leftarrow \langle mp \rangle} \left[E_3(x_1^1, x_2^1) \wedge E_4(x_1^1 + x_2^1)\right] \cdot (1 \pm p)^2 \end{aligned} \tag{10}$$

Putting together (8-10), we get:

$$\mathbf{E}\left[C_{\bar{\ell}}\right] \in p^2 \cdot (1 \pm p)^4 \cdot \Pr_{x_1^1, x_1^2 \leftarrow \langle mp \rangle} \left[E_3(x_1^1, x_2^1) \wedge E_4(x_1^1 + x_2^1)\right] \tag{11}$$

The probability expression above is similar to the one we started with in (8), except that it is for the case of $k = 2$ and the range $\langle mp \rangle$ instead of $k = 4$ and range $\langle m \rangle$. For general k, we get a similar recursive expression in terms of $k/2$ that enables us to compute the overall bound efficiently, both analytically and computationally. In the present case, we simply have to apply (4) and (7) in turn once more to get the following final bound:

$$\mathbf{E}\left[C_{\bar{\ell}}\right] \in p^3 \cdot (1 \pm p)^6 \cdot \frac{1}{mp^2} \subseteq p^4 \cdot (1 \pm O(p)) \tag{12}$$

The above is again quite close to its heuristic evaluation in (2). The expectation of C can then be computed as in (3) to get:

$$\mathbf{E}\left[C\right] \in c^4 \cdot (1 \pm O(p)) \tag{13}$$

The upper bound on expectation above already gives us an upper bound on the success probability of the k-Tree algorithm using the Markov inequality. It remains, then, to show a corresponding lower bound.

Concentration. Next we move on to proving that (a generalization of) assumption (a) is valid. Our hope here is to show that when the expectation of C is significantly larger than 0, then with somewhat high probability, we have $C \neq 0$ – that is, the algorithm succeeds. Clearly, this is not true of arbitrary random variables – if C was 0 with probability $(1 - n^{-4})$ and n^4 with probability n^{-4}, it would still have an expected value of 1. So to show this, we will need to rely on additional properties of C.

The property we will use is that C is the sum of the n^4 identically distributed indicator random variables $C_{\bar{\ell}}$. If this set of random variables had been independent, we could have used a Chernoff-Hoeffding bound to show that C strongly concentrates around its expectation, and thus if its expectation is sufficiently larger than 0, then it will take non-zero values with large probability. However, these variables are not independent.

Consider, for example, tuples of indices $\bar{\ell} = (\ell_1, \ldots, \ell_4)$ and $\bar{\ell}' = (\ell'_1, \ldots, \ell'_4)$ that differ only on the value of their fourth element. If $C_{\bar{\ell}} = 1$, this implies that $(L_1[\ell'_1] + L_2[\ell'_2]) = (L_1[\ell_1] + L_2[\ell_2]) \in \langle mp \rangle$. Thus, the tuple $(L_1[\ell'_1], \ldots, L_4[\ell'_4])$ already passes one of the filters of the algorithm, and so $C_{\bar{\ell}'}$ is now more likely to be 1 than it would have been without conditioning on $C_{\bar{\ell}} = 1$. In fact, this example illustrates that the set of random variables $\{C_{\bar{\ell}}\}$ are not even pairwise independent. This lack of independence is perhaps the most significant challenge in analyzing the performance of the k-Tree algorithm.

Our way around this is the following set of observations. The degree of correlation between $C_{\bar{\ell}}$ and $C_{\bar{\ell}'}$ is proportional to the number of co-ordinates on

which $\bar{\ell}$ and $\bar{\ell}'$ agree. For instance, if $\bar{\ell}$ and $\bar{\ell}'$ do not agree on any co-ordinate, then $C_{\bar{\ell}}$ and $C_{\bar{\ell}'}$ are, in fact, independent. Fortunately, for $t \in [0, 4]$, the number of pairs $\bar{\ell}$ and $\bar{\ell}'$ that agree on t co-ordinates decreases as t (and thus the correlation) increases. Out of the n^8 possible pairs, only $O(n^7)$ pairs agree on 1 co-ordinate, $O(n^6)$ agree on 2, and $O(n^5)$ agree on 3.

Taking advantage of this, we are able to carefully bound the second moment of C. Then, we use second-moment-based concentration bounds to show that if the expectation of C is large enough, it will indeed be non-zero with substantial probability.

Computing the Second Moment. The second moment of C can be written as follows:

$$\mathbf{E}\left[C^2\right] = \sum_{\bar{\ell},\bar{\ell}' \in [n]^4} \mathbf{E}\left[C_{\bar{\ell}} \cdot C_{\bar{\ell}'}\right] = \sum_{\bar{\ell},\bar{\ell}' \in [n]^4} \Pr\left[C_{\bar{\ell}} = 1 \wedge C_{\bar{\ell}'} = 1\right] \tag{14}$$

Due to symmetry, the term corresponding to any $\bar{\ell}$ and $\bar{\ell}'$ in the sum above is fully determined by the set of co-ordinates that $\bar{\ell}$ and $\bar{\ell}'$ agree on (that is, it does not matter what specific values the ℓ_i's and ℓ'_i's take once the set of i's on which they agree is determined). To capture this, we define the string $\delta(\bar{\ell}, \bar{\ell}') \in \{0,1\}^4$ whose i^{th} co-ordinate is defined as follows:

$$(\delta(\bar{\ell}, \bar{\ell}'))_i = \begin{cases} 0 & \text{if } \ell_i = \ell'_i \\ 1 & \text{if } \ell_i \neq \ell'_i \end{cases}$$

We then segregate the pairs $(\bar{\ell}, \bar{\ell}')$ in the sum in (14) according to the value of $\delta(\bar{\ell}, \bar{\ell}')$. For any $\mathbf{b} \in \{0,1\}^4$, the the number of such pairs with $\delta(\bar{\ell}, \bar{\ell}') = \mathbf{b}$ is $n^4(n-1)^{wt(\mathbf{b})} \approx n^{4+wt(\mathbf{b})}$, where $wt(\mathbf{b})$ is the Hamming weight of $\mathbf{b}$. We can then re-write the second moment as follows:

$$\begin{aligned} \mathbf{E}\left[C^2\right] &= \sum_{\mathbf{b} \in \{0,1\}^4} \sum_{\substack{\bar{\ell},\bar{\ell}' \in [n]^4 \\ s.t.\ \delta(\bar{\ell},\bar{\ell}') = \mathbf{b}}} \Pr\left[C_{\bar{\ell}} = 1 \wedge C_{\bar{\ell}'} = 1\right] \\ &\approx \sum_{\mathbf{b} \in \{0,1\}^4} n^{4+wt(\mathbf{b})} \cdot f(\mathbf{b}) \end{aligned} \tag{15}$$

where $f(\mathbf{b})$ is defined to be the probability that $C_{\bar{\ell}} = C_{\bar{\ell}'} = 1$ for any $\bar{\ell}$ and $\bar{\ell}'$ such that $\delta(\bar{\ell}, \bar{\ell}') = \mathbf{b}$. In computing $f(\mathbf{b})$, our broad approach is similar to the one we took when computing the first moment – to repeatedly replace intermediate list elements with uniformly random values while computing the probabilities that the filters of the algorithm are passed. Only now, we need to do this simultaneously for two partially dependent executions of the algorithm.

We start first with the events E_1 as defined above, which capture the first filter applied to the first two elements in the lists. The probability that this event happens in both executions is:

$$\Pr_{L_1, L_2}\left[(L_1[\ell_1] + L_2[\ell_2]) \in \langle mp \rangle \wedge (L_1[\ell'_1] + L_2[\ell'_2]) \in \langle mp \rangle\right] \tag{16}$$

Let $\mathbf{b} = \delta(\bar{\ell}, \bar{\ell}')$, and denote its bits by $\mathbf{b}_1, \ldots, \mathbf{b}_4$. If $\mathbf{b}_1 = \mathbf{b}_2 = 0$, then $\ell_1 = \ell'_1$ and $\ell_2 = \ell'_2$, meaning the two events above are the same and so by (9) the above probability is $\approx p$ (ignoring factors of $(1 \pm O(p))$ for now). Similarly if $\mathbf{b}_1 = \mathbf{b}_2 = 1$, the two events are independent and the probability is $\approx p^2$. If exactly one of $\mathbf{b}_1$ and $\mathbf{b}_2$ is 1, then again the probability is roughly $\leq p^2$, due the following fact, which again follows from elementary computations:

$$\Pr_{w,x,y \leftarrow \langle s \rangle} [w + x \in \langle sp \rangle \wedge w + y \in \langle sp \rangle] \leq p^2 \cdot \left(1 + O\left(\frac{1}{sp}\right)\right)^2 \tag{17}$$

Overall, we have approximately the following (again ignoring $(1 \pm O(p))$ factors):

$$\Pr_{L_1, L_2} [E_1(L_1[\ell_1], L_2[\ell_2]) \wedge E_1(L_1[\ell'_1], L_2[\ell'_2])] \leq p^{1+\max(\mathbf{b}_1, \mathbf{b}_2)} = p^{1+\mathbf{b}_1^1} \tag{18}$$

where we define $\mathbf{b}_1^1$ to be $\max(\mathbf{b}_1, \mathbf{b}_2)$.

Next, we look at the joint distribution of the sums $(L_1[\ell_1] + L_2[\ell_2], L_1[\ell'_1] + L_2[\ell'_2])$. If $\mathbf{b}_1^1 = 0$, then these two sums are equal, and by (6) are each $(1 + O(p))$-close to being uniformly distributed over $\langle sp \rangle$. If $\mathbf{b}_1 = \mathbf{b}_2 = 1$, then these are independent and so the joint distribution is now close to being uniform over $\langle mp \rangle \times \langle mp \rangle$. We show by computing the MR distances explicitly that even in the case where exactly one of $\mathbf{b}_1$ and $\mathbf{b}_2$ is 1, this joint distribution is close to being uniform over $\langle mp \rangle \times \langle mp \rangle$.

So overall, if $\mathbf{b}_1^1 = 0$, this pair of sums is equal with close-to-uniform marginals, and if $\mathbf{b}_1^1 = 1$, the pair is close to being independently uniformly distributed over $\langle mp \rangle$. Notice that this is exactly the relationship that the pair $(L_1[\ell_1], L_1[\ell'_1])$ had to $\mathbf{b}_1$, or $(L_2[\ell_2], L_2[\ell'_2])$ had to $\mathbf{b}_2$, except that the marginal distributions there were over $\langle m \rangle$. The same sequence of arguments can be made with the events E_2, where in place of $\mathbf{b}_1^1$ we use $\mathbf{b}_2^1 = \max(\mathbf{b}_3, \mathbf{b}_4)$. In effect, this lets us set up a recursive argument where we can replace the intermediate sums with uniform elements from $\langle mp \rangle$, and the string $\mathbf{b}$ with $(\mathbf{b}_1^1, \mathbf{b}_2^1)$.

We can then repeat the entire argument above once more (essentially with $k = 2$ now), with $\mathbf{b}_1^2 = \max(\mathbf{b}_1^1, \mathbf{b}_2^1)$, to show that the probability of each of the pairs of events corresponding to E_3 and E_4 happening is roughly bounded by $p^{1+\mathbf{b}_1^2}$. Overall, we end up with the following bound:

$$\begin{aligned} \Pr[C_{\bar{\ell}} = 1 \wedge C_{\bar{\ell}'} = 1] = f(\mathbf{b}) &\leq p^{1+\mathbf{b}_1^1} \cdot p^{1+s\mathbf{b}_2^1} \cdot p^{1+\mathbf{b}_1^2} \cdot p^{1+\mathbf{b}_1^2} \cdot (1 + O(p)) \\ &= p^{4+\mathbf{b}_1^1+\mathbf{b}_2^1+2\mathbf{b}_1^2} \cdot (1 + O(p)) \end{aligned} \tag{19}$$

Combining this bound with (15), we get:

$$\begin{aligned} \mathbf{E}\left[C^2\right] &\leq \sum_{\mathbf{b} \in \{0,1\}^4} n^{4+wt(\mathbf{b})} \cdot p^{4+\mathbf{b}_1^1+\mathbf{b}_2^1+2\mathbf{b}_1^2} \cdot (1 + O(p)) \\ &= c^4 \cdot (1 + O(p)) \cdot \sum_{\mathbf{b} \in \{0,1\}^4} c^{wt(\mathbf{b})} \cdot p^{\mathbf{b}_1^1+\mathbf{b}_2^1+2\mathbf{b}_1^2-wt(\mathbf{b})} \end{aligned} \tag{20}$$

Using graph-theoretic arguments, we can show that for any $\mathbf{b} \notin \{0^4, 1^4\}$, the difference $(\mathbf{b}_1^1 + \mathbf{b}_2^1 + 2\mathbf{b}_1^2 - wt(\mathbf{b}))$ is at least 1, and so $p^{\mathbf{b}_1^1+\mathbf{b}_2^1+2\mathbf{b}_1^2-wt(\mathbf{b})} \leq p$. For $\mathbf{b} \in \{0^4, 1^4\}$, this difference is 0. So we can bound the above sum as follows:

$$\begin{aligned}\sum_{\mathbf{b}\in\{0,1\}^4} c^{wt(\mathbf{b})} \cdot p^{\mathbf{b}_1^1+\mathbf{b}_2^1+2\mathbf{b}_1^2-wt(\mathbf{b})} &\leq 1 + c^4 + \sum_{\mathbf{b}\in\{0,1\}^4\setminus\{0^4,1^4\}} c^{wt(\mathbf{b})} \cdot p \\ &\leq 1 + c^4 + p \cdot \sum_{\mathbf{b}\in\{0,1\}^4} c^{wt(\mathbf{b})} \\ &= 1 + c^4 + p \cdot (1+c)^4 \end{aligned} \tag{21}$$

Overall, from (20) and (21), we get the following bound on the second moment:

$$\mathbf{E}\left[C^2\right] \leq c^4 \cdot \left(1 + c^4 + p \cdot (1+c)^4\right) \cdot (1 + O(p)) \tag{22}$$

Computing the Lower Bound. With the bounds on the first two moments from (13) and (22), the required lower bound can be obtained using the Paley-Zygmund inequality, which states that for any non-negative random variable Z and any $\theta \in [0, 1]$:

$$\Pr\left[Z > \theta\, \mathbf{E}\left[Z\right]\right] \geq (1-\theta)^2 \frac{\mathbf{E}\left[Z\right]^2}{\mathbf{E}\left[Z^2\right]}$$

Applying this bound to C with $\theta = 0$, we get:

$$\begin{aligned}\Pr\left[k\text{-Tree succeeds}\right] = \Pr\left[C > 0\right] &\geq \frac{\mathbf{E}\left[C\right]^2}{\mathbf{E}\left[C^2\right]} \\ &\geq \frac{c^8 \cdot (1 - O(p))}{c^4(1 + c^4 + p(1+c)^4)(1 + O(p))} \\ &\geq \frac{c^4}{1 + c^4 + p \cdot (1+c)^4} \cdot (1 - O(p)) \end{aligned} \tag{23}$$

Tightening the Bound. The above approach naturally generalizes to any k that is a power of 2 to give the following lower bound:

$$\Pr\left[k\text{-Tree succeeds}\right] \geq \frac{c^k}{1 + c^k + p \cdot (1+c)^k} \cdot (1 - O(kp))$$

While their asymptotics for any fixed k are the same, for moderately large values of k and some reasonable concrete values of m and n, the above bound ends up being much weaker than the lower bound actually stated in Theorem 1. This is because, even when $c = 1$, the $p(1+c)^k$ term in the denominator quickly starts to dominate. The bound stated in Theorem 1 remains meaningful for a wider range of concrete values of the parameters, and is obtained by performing a more careful analysis of the sum in (20), using a recursive argument to bound the entire sum together rather than each term separately.

The constants in the statement of Theorem 1 are obtained by carefully tracking the constants that come up in the course of the above analysis. Even so, the constants stated there are sub-optimal due to limits on human tolerance, and the algorithmic implementation of this proof (as captured in Theorem 2) improves upon them significantly.

4 Related Work

The worst-case version of the k-SUM problem has been studied extensively in the literature on algorithms [29, 6, 18, 15, ...], data structures [35, 26, 16, ...], and complexity theory [24, 7, 48, 14, 25, 22, 5, 42, 43, 28, 1, ...]. In the study of the fine-grained complexity of problems within P, in particular, connections have been discovered between the complexity of this problem and those of various important problems from computational geometry, graph theory, data structures, etc. [24, 7, 48, 42, 3, 34]. A simple meet-in-the-middle algorithm solves this problem in $\tilde{O}(n^{\lceil k/2 \rceil})$ time [29]. The best algorithms known are faster than this by only a small polylog factor [6, 28, 15], or run in time $\tilde{O}(m+n)$ [12, 31]. It has been conjectured that the best worst-case algorithm for this problem runs in time $n^{\lceil k/2 \rceil - o(1)}$ [24, 2].

For average-case k-SUM, non-trivial algorithms (beyond known worst-case algorithms) have so far been restricted to Wagner's k-Tree algorithm [49] and its extensions to using smaller lists (at the cost of running time) [40], to values of k that are not power of 2 [41, 19], to generalizations with better time-memory tradeoffs [41, 9, 19], and to the quantum setting [27]. These algorithms have found extensive use across cryptography and cryptanalysis [46, 49, 21, 36, 8, ...]. They are also closely related to some of the best algorithms for the Learning Parity with Noise problem [10].

A particularly important application of k-Tree over integers emerged in cryptanalysis of signature schemes. Several discrete logarithm-based blind signatures relied on the hardness of the ROS problem [23]. Wagner [49] showed that k-Tree over $\mathbb{Z}_p$ could efficiently solve this problem, impacting the security of these signature schemes including Schnorr [46] and Okamoto-Schnorr [44] signatures. This application witnessed a significant advancement with Benhamouda et al.'s [8] breakthrough, who combined early-terminated k-Tree with novel techniques to achieve polynomial-time attacks when enough concurrent signing sessions are available. Their work demonstrates that precise bounds on k-Tree's performance over integers are crucial for understanding concrete security guarantees of these signature schemes. This prevalence of k-Tree over $\mathbb{Z}_p$ in cryptanalysis motivates our focus on providing rigorous bounds for the integer variant of the algorithm.

Lattice-based conditional lower bounds are known to indicate that the k-Tree algorithm for k-SUM is optimal in its asymptotic dependence on k [11]. Some conditional lower bounds are also known for the complexity of the k-SUM problem given lists of size close to $m^{1/k}$ [20, 4].

Comparison with Existing Analyses. As noted earlier, Wagner's original argument [49] of the correctness of the k-Tree algorithm was heuristic and relied on assumptions regarding the distribution and independence of elements in intermediate lists computed by the algorithm. The conclusion of this argument was that the algorithm works with constant success probability when given input lists of size $m^{1/(\log k+1)}$, and that in this case it runs in time $O(k \cdot m^{1/(\log k+1)})$.

This argument was made rigorous by Minder and Sinclair [40] for the variant of the k-Tree algorithm that sovles the k-SUM problem over the group $\mathbb{Z}_2^m$ (in which case it is referred to as the k-XOR problem). In this case, Wagner's assumption regarding the uniform distribution of intermediate list elements is immediately seen to be true. Minder and Sinclair also follow the approach (described in Section 3) of then computing the first two moments of the random variable counting the number of solutions found, and then using these to prove concentration bounds. In the case of k-XOR, the process of computing these moments turns out to be much simpler due to the fact that the XOR of any arbitrary vector with a uniformly random vector results again in a uniformly random vector. Some of the questions that come up during this analysis of k-Tree over $\mathbb{Z}_2^m$ also come up in the analysis of an approach to hashing called Simple Tabulation Hashing (see, for example, [45]), but the implications of the results there for Minder and Sinclair's analysis are not immediately clear to us.

The first rigorous analysis of the k-Tree algorithm over integers[7] was by Lyubashevsky [39], who showed that the algorithm works with high probability if the input lists are of size at least $\Omega(k^2 \log k \cdot m^{2/\log k})$. This was improved by Shallue [47] to lists of size at least $\Omega(k \cdot m^{1/\log k})$. Both of these actually study a variant of the k-Tree algorithm where the factor by which the permitted range shrinks at each level of the tree is set to $p = m^{-1/\log k}$ instead of $m^{-1/(\log k+1)}$. In this case, the final list is only permitted to contain 0's, and when the input list size is $m^{1/\log k}$, the expected number of solutions found by the algorithm is also $\Theta(m^{1/\log k})$. Both Lyubashevsky and Shallue were interested in using the k-Tree algorithm for very large values of k, and so the difference between $\log k$ and $(\log k + 1)$ in the exponent were not significant in their applications. It is possible that Shallue's analysis can be made to work for the original k-Tree algorithm, but this is to be verified and even then it is not clear whether the bounds will work for list sizes smaller than $k \cdot m^{1/(\log k+1)}$.

Both of the above papers take similar approaches in their analysis, and we briefly describe Shallue's here. Similar to us, Shallue starts by showing that the elements in the intermediate lists are close to uniform; the distance measure used there is also an element-wise bound on differences in probability mass, but while we use the MR distance to measure relative difference, they measure absolute difference. We believe our choice is better for concrete parameters, but asymptotically these choices are likely equivalent. The larger difference is in

[7] We continue to ignore the distinction between k-Tree over integers and over $\mathbb{Z}_m$, following the discussion in Remark 4.

how they deal with the dependence between list elements. They start with the observation that even though there are dependencies between list elements, it is possible to find a large enough subset of elements in each list such that all correlations among them are relatively small. They then use martingale-based concentration bounds to recursively bound the size of each intermediate list.

Their utilization of correlation bounds on larger sets of variables eventually results in a better dependence of the probability bound on the list-size overhead. They show that if the input lists are of size $c \cdot m^{1/\log k}$ for some $c \geq k$, then the probability of failure is roughly at most $(m^{1/\log k} e^{-c/1024})$, whereas our bound for lists of size $c \cdot m^{1/(\log k+1)}$ is around c^{-k}. For very large values of c, the former will be much smaller, but for moderate values $c = \Theta(1)$, the latter bound is better.

A different approach to analyzing the k-Tree algorithm was taken by Joux, Kippen, and Loss [33]. They get around the uniformity and independence issues by modifying the k-Tree algorithm in such a way that these properties actually hold as assumed by the heuristic analysis. This makes the algorithm slightly worse than the original, but easier to analyze. They then show that this modified algorithm works with probability roughly larger than $(1 - e^{-\mathrm{poly}(k)})$ if the input list sizes are $n = \beta(k) \cdot m^{1/(\log k+1)}$, where $\beta(k) \approx (1.26)^{\frac{\ell-1}{\ell+1}} \cdot 6^{\frac{(\ell-1)(\ell+2)}{(\ell+1)}}$, with $\ell = \log k$. Further, their modified algorithm runs in time $\Theta(kn)$ in this case. This again results in very good bounds on the probability of success, but the bounds only work for large values of c, especially for larger values of k – for instance, $\beta(4) \approx 3.563$ and $\beta(1024) \approx 7964$.

In summary, the following are the most significant points of comparison between our results and the existing analyses of Shallue [47] and Joux et al. [33] of the k-Tree algorithm over integers:

- The results in the other papers do not give meaningful bounds for $c \leq 1$ – that is, when the size of the lists is $n = m^{1/(\log k+1)}$, as suggested by Wagner's heuristic analysis, or smaller – while ours do. This is also true for some range of values of c larger than 1, with the range depending on k.
- For sufficiently larger list sizes ($c \gg 1$), the approaches in the other papers can likely show lower bounds on the success probability that are much closer to 1 than our bounds can. This gap can be closed using certain optimizations to the k-Tree algorithm as described in Remark 2, though that does not vindicate our analysis itself.

We present comparisons of our results with these in Figure 4, making optimistic assumptions regarding the constants in the bounds of the other papers, as well as regarding the regime of their validity, wherever relevant.

The practical significance of this improved precision is evident in contemporary cryptographic applications. For instance, in the context of attacks leveraging the k-Tree algorithm, such as the ROS attack detailed by Benhamouda et al. [8], a more accurate understanding of the algorithm's performance directly impacts

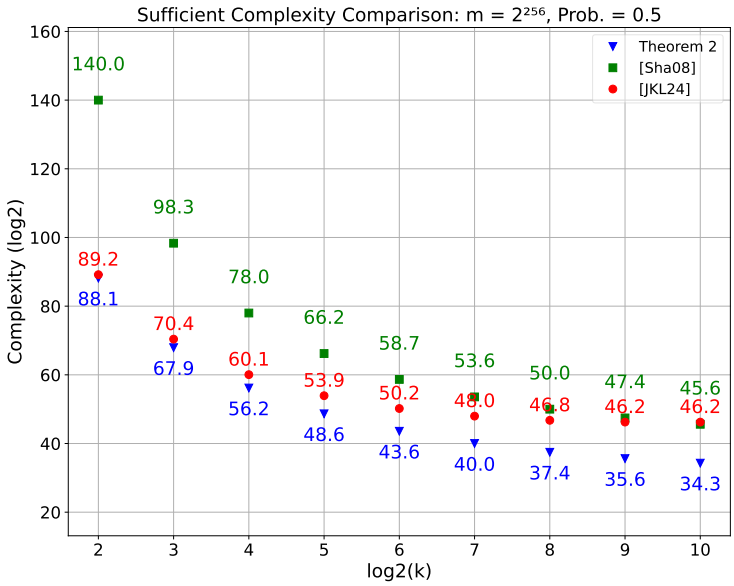


Fig. 4: Comparison of bounds produced by Theorem 2 with those of [47] and [33]. For the other papers, we plot the minimum complexity of k-Tree for which they show non-trivial lower bounds on success probability (usually close to 1). In our case, we plot the smallest complexity of k-Tree for which the lower bound on success probability produced by Theorem 2 is at least 0.5.

the assessment of security margins. Our analysis provides a refined tool for such evaluations. As a specific example, for parameters $m = 2^{256}$ and $k = 32$ (where $\log_2(k) = 5$), our analysis yields a complexity of approximately 2^{49}. This is a notable improvement over estimates from Joux et al.'s analysis [33] (2^{54}), and Shallue's analysis [47] (2^{66}). This tighter bound, also reflected in broader comparisons like those in Figure 4, allows for more precise evaluations across a wider range of parameter regimes than previously possible.

5 Computed Bounds

The theoretical analysis presented in Section 3 and the Analysis section in the paper's full version provides asymptotically tight bounds for the k-Tree algorithm. However, for some practical parameter values that modern computers can handle, these bounds may not be as precise as one might desire due to the approximations used in the proofs. While these approximations do not affect the asymptotic tightness, they can have significant impact on the bounds for the parameter ranges we wish to evaluate empirically.

To address this limitation, we have implemented a set of computer programs that compute these bounds without the approximations used in the theoretical

analysis. This approach allows us to obtain much tighter bounds for smaller parameter values, which are particularly relevant for practical applications and our experimental evaluations.

By removing the approximations used in the manual proofs, these computational methods provide a bridge between our asymptotic analysis and the actual performance of the algorithm. The bounds computed by these programs serve two crucial purposes:

- Offer more accurate predictions of the algorithm's behavior for practical parameter ranges.
- Provide a means to validate our theoretical predictions against experimental measurements.

Due to page limitation, the detailed pseudo-code and theoretical justifications are provided in Computed Bounds section in the paper's full version.

Visualizations and Interpretations. We present examples of our computing bounds for various parameter settings. These visualizations help us better understand the tightness of the upper and lower bounds of the success probability as key parameters, such as m and k, change. We will also directly compare our bounds with our experimental results to validate the trends we observe.

Success Probability Bounds. In Figure 5, the most notable observation is that for a fixed m, the lower bounds on the success probability become looser as k increases. This trend will be further illustrated when we later compare the bound with our empirical measurements of success probabilities. Such phenomenon is clearly visualized in the case where $m = 2^{64}$ and $k = 128$ (see Figure 5b). In this example, the lower bound flattens early and is far from 1, indicating a quick relaxation of the bound as k increases. This behavior shows that when k is large, the success probability bounds are less effective for moderately small m.

However, as m increases, the bounds become significantly tighter for all values of k. For instance, in the case where $m = 2^{256}, k = 1024$ (Figure 5d), the bounds exhibits much better tightness compared to when $m = 2^{64}$ and $k = 128$. This improvement in the tightness of the bounds align with our conclusion in Section 1 that our bounds become asymptotically tight as m approaches infinity.

Complexity Bounds. In the examples shown in Figure 6, we present the sufficient and necessary complexities (total list sizes) for achieving a target success probability. For the sufficient complexity, given a fixed m, we compute the minimum input list size via binary search such that our success probability lower bound equals or slightly larger than the specified probability threshold. We then use this minimum input list size as an input to compute our upper bound on the expected total size of all lists. For the necessary complexity, we first search for the minimum input list size such that our success probability upper bound equals or slightly smaller than the specified probability threshold. We then use

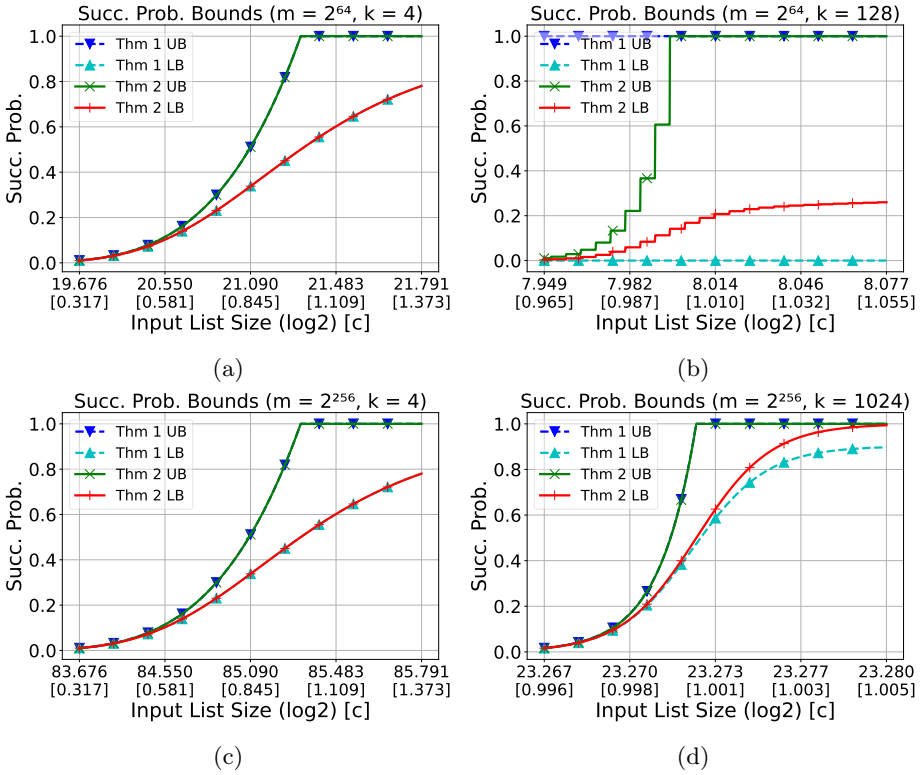


Fig. 5: Success probability upper and lower bounds when varying the input list size under fixed m's and k's. The values in the square brackets report $c = n/m^{1/(\log k+1)}$ as described in Theorem 1. Note that the cascading patterns of the bounds for certain values of m and k are due to rounding the input list size to the nearest integer.

this minimum input list size to compute our lower bound on the expected total size of all lists.

As observed in these figures, when k increases, the upper bound on the total list size first decreases accordingly then increases again as k continues to grow. This trend is more pronounced for small m's, as shown in Figure 6a. This behavior is consistent with the results shown in Figure 5, where the lower bounds on the success probability become looser as k increases. The immediate consequence is that the input list size has to be substantially larger to achieve the same success probability when k is large. That accounts for the surge of total list size observed in Figure 6a. As m increases, the total list sizes become more stable. In our experiments, we see a similar trend of decrease-then-increase of the complexities in our empirical measurements.

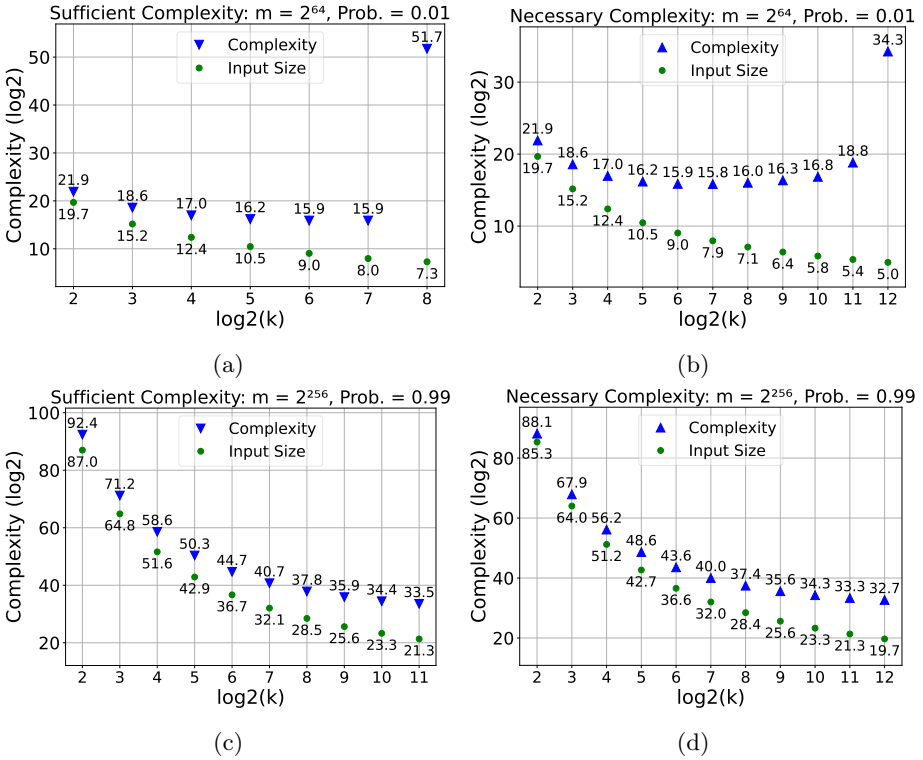


Fig. 6: Size upper and lower bounds when varying k for fixed m's (the fewer amount of k in some of the Sufficient Complexity plots is due to the lower bound cannot reach the probability threshold even for very large c). In (d), its similarity with Figure 4 is due to precision limits in plotting the figures.

6 Experiments

The theoretical analysis and bounds computed in the previous sections are to offer insights into the expected performance of the kTree algorithm. However, empirical experiments are crucial to understand the algorithm's behavior under practical conditions and to validate these theoretical predictions. In this section, our primary objectives are as follows:

1. Evaluate the behavior of the kTree algorithm under various parameter configurations;
2. Compare the empirical results with our computed bounds.

The results we demonstrate in this section provide empirical evidence on how different parameters influence the algorithm's success probability, time, and space complexity.

6.1 Experimental Setup

In discussions below, we use symbols defined in the context of the description of the kTree algorithm in Figure 1.

Algorithm Implementation. We implemented the kTree algorithm in C++, closely following the description provided in Figure 1. The Merge subroutine was implemented by conducting a binary search over the sorted second list for each element in the first list.

Parameter Configurations. We investigate the impact of varying the following parameters:

- m: taking values from $\{2^{64}, 2^{96}, 2^{128}\}$. Due do limitations in computational resources, we were unable to explore larger values of m.
- k: The number of input lists, with values ranging from 4 to 1024, varied in powers of 2. Note that not all k's are feasible for all m's. For example, when $m = 2^{128}$, any k smaller than 512 results in a large input list size that exceeds the computational resources available. We report all the feasible results we have obtained.
- n: The size of each input list, dynamically adjusted via binary search to achieve target success probabilities varied from 0.01 to 1.0. Notice that not all the target probabilities are feasible due to either the nature of the problem (e.g., when m is relatively small and k is large, increasing the input list size by 1 will increase the success probability from below 0.2 to above 0.8) or the limit of computational resources. We report all the feasible results we have obtained.

Evaluation Metrics.

- **Success Probability:** The fraction of trials where the algorithm successfully identifies a set of indices such that the indexed elements of the k input lists sum to 0.
- **Total Size**: The sum of the sizes of all lists generated when running the kTree algorithm, measured in terms of the number of elements:

$$\sum_{d=0}^{\log k} \sum_{i \in \left[\frac{k}{2^d}\right]} \left|L_i^d\right|.$$

  We use this measurement as a proxy for the time complexity of the algorithm (see Remark 3).
- **Max Level Size**: The maximum sum of the sizes of all lists generated at one level when running the kTree algorithm:

$$\max_d \sum_{i \in \left[\frac{k}{2^d}\right]} \left|L_i^d\right|.$$

  This measurement corresponds to the space complexity of the algorithm.

Hardware and Software Environment. All experiments were conducted on a machine running Ubuntu 22.04 LTS (64-bit) system on an Intel Core i7-12700 CPU @ 4.90GHz with 64 GB of RAM. The kTree algorithm was implemented in C++ 17, and the experimental results were managed and analyzed using Python 3.9 with libraries such as NumPy and Matplotlib for plotting results.

6.2 Success Probability

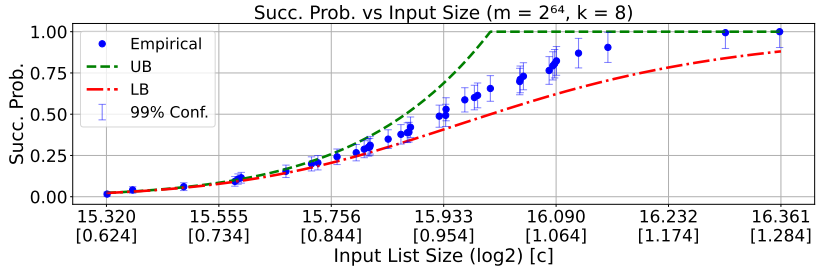
This section is to explore and measure the effect of varying the input list size on the success probability of the kTree algorithm under different configurations of m and k.

For each combination of m and k, we conducted a series of trials with varying input list sizes. We started with a list size corresponding to $c = 1$, where $c = n/m^{1/(\log k+1)}$ as defined in Theorem 1. We then used binary search to iteratively adjust the list size, exploring a range of c values that span a typical set of success probabilities (i.e., percentage points). For each parameter configuration, we performed 1000 independent trials to calculate the empirical success probability. Then we further compare the empirical results with our theoretical bounds.

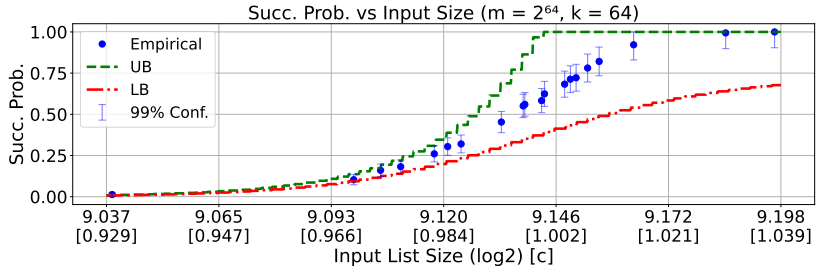
Figure 7 presents the results of our experiments. The blue dots represent the measured success probability, while the dashed red and green lines represent the upper and lower bounds derived from our theoretical analysis, respectively. The error bars around the empirical measurements indicate the 99% confidence interval, calculated using the Chernoff inequality. The parameter c, as defined earlier, serves as a normalized measure of the input list size relative to m and k. It allows us to compare results across different parameter configurations and relates directly to our theoretical bounds. These plots enable us to visualize how the success probability transitions from near 0 to near 1 as the input list size increases, and how this transition varies for different values of m and k. They also allow us to assess the tightness of our theoretical bounds under various conditions.

The results shown in Figure 7 illustrate several key trends in the success probability of the kTree algorithm as the input list size varies. We report both the actual input list size n and the corresponding c value. We make the following general observations:

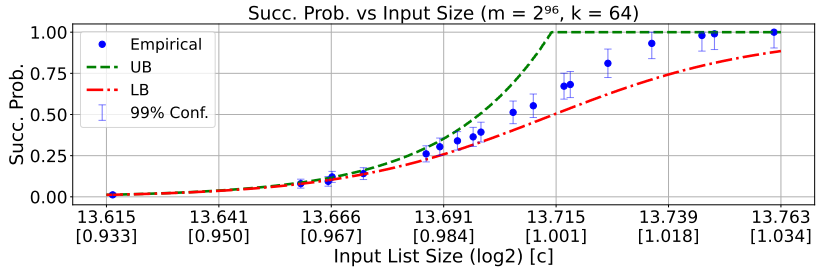
- Generally, after fixing reasonable values for m and k, the actual success probability approaches 0 and 1 at the left and right extremes when the c value is slightly below or above 1.
- For a fixed m, as k increases, the success probability converges to 0 and 1 more rapidly as c is decreased or increased linearly.
- In terms of the bounds we provided, for a fixed m, the tightness of the bounds improves as k decreases. Conversely, for a fixed k, increasing m significantly improves the tightness of our bounds, which aligns with the earlier conclusion that our bounds are asymptotically tight.



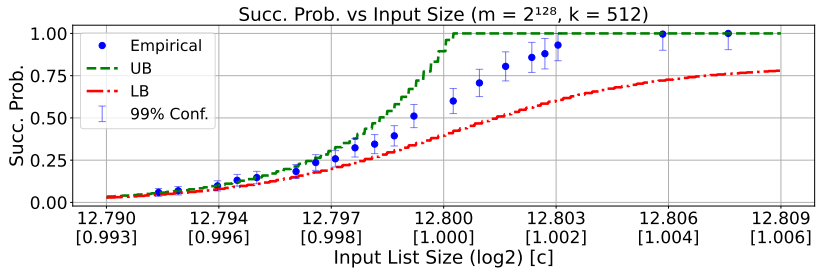
(a)



(b)



(c)



(d)

Fig. 7: Success probability when varying the input list size under fixed m's and k's. The values in the square brackets report $c = n/m^{1/(\log k+1)}$ as described in Theorem 1.

As expected from our theoretical model, the c value has a direct and significant impact on the success probability. When c is below 1, the success probability approaches 0 gradually. Conversely, when c exceeds 1, the success probability converges to 1. When m is fixed, increasing k accelerates the convergence of the success probability toward 0 and 1. Larger k increases the rate at which the success probability transitions between failure and success. This faster convergence for larger k highlights a sharper threshold between the regions where the algorithm succeeds and fails, which could be beneficial in applications where precise control over success rates is desired.

The empirical results closely align with our theoretical predictions regarding the behavior of the kTree algorithm. Specifically, the success probability's dependence on the c value and the role of k in determining the speed of convergence are both well-captured by the theoretical bounds. The observation that the bounds are tighter for smaller k (for fixed m) and for larger m (for fixed k) is consistent with the asymptotic analysis provided earlier. Moreover, the empirical data validates the hypothesis that our bounds are asymptotically tight: as m grows relative to k, the bounds become increasingly accurate, which confirms that the algorithm's performance can be effectively predicted using the theoretical framework developed.

6.3 Complexity

The primary objectives of this section are to measure and analyze the time and space complexity of the kTree algorithm under various parameter configurations. Specifically, we aim to:

- Observe how the total list size (a proxy for time complexity) and maximum level size (space complexity) vary with different values of k for fixed m and success probability.
- Investigate the impact of different success probability thresholds on the algorithm's complexity.
- Identify optimal parameter configurations that minimize time, space usage.

For each m (2^{64} and 2^{96}) and success probability threshold (0.01 and 0.99), we varied k from 4 to 1024 in the powers of 2. For each configuration:

1. We used binary search to find the minimum input list size n that achieves a success probability above the specified threshold.
2. We measured the total size and the maximum level size to represent the time and space complexity.
3. We conduct 1000 trials and report the average and standard deviation to ensure reliability.

Figure 8 provide a detailed analysis of the algorithm's complexities. For fixed m and a given probability threshold, we observe that both time and space complexities initially decrease and then increase as k grows. This reflects a trade-off: from the results we observed in Figure 7, increasing k boosts the value of

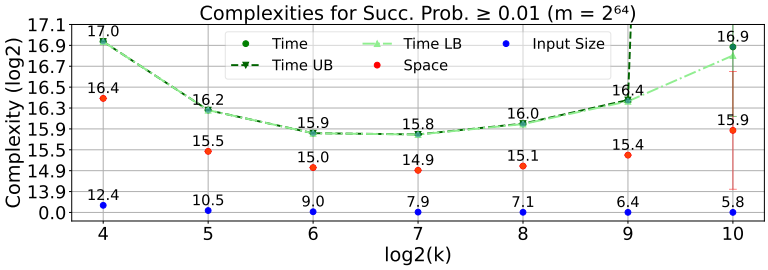
$p = m^{\frac{-1}{\log k+1}}$ and significantly decrease the threshold $c = np$ that achieve success probability 1. This further leads to a drastically decrease in the input list size. Nevertheless, at the same time, a larger k increases the number of total lists and the number of levels in the kTree algorithm. This indicates the importance of finding an optimal k value that minimizes complexity.

The impact of the probability threshold (0.01 vs 0.99) on the optimal complexity is relatively small, particularly for larger k values. This suggests that achieving a constantly higher success probability doesn't necessarily incur a significant additional cost. This can also be inferred from the analytical bounds, where the success probability changes exponentially (where k is the exponent) as n vary.

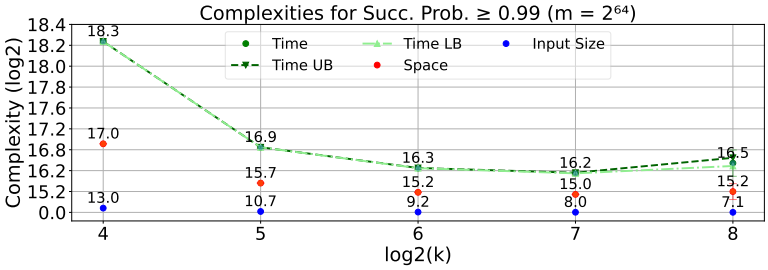
We also observe that as m increases, the minimum complexity (in both time and space) occurs at a higher value of k. This indicates that for larger m, the optimal choice of k—in terms of minimizing both time and space complexity—shifts upwards. This implies that for larger problem sizes, using more input lists can be beneficial up to a certain point. And the insight we obtained here allows users of the algorithm to select the most efficient parameter configuration using our bounds.

Acknowledgements

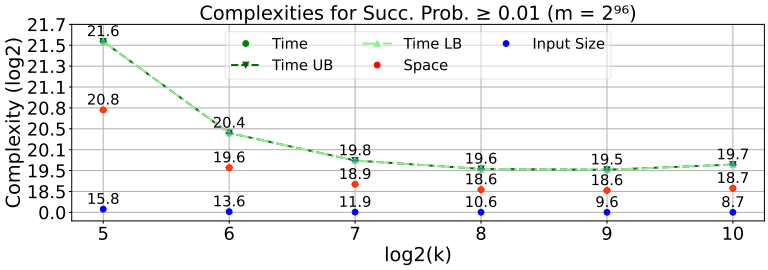
This work was supported by the National Research Foundation, Singapore, under its NRF Fellowship programme, award no. NRF-NRFF14-2022-0010.



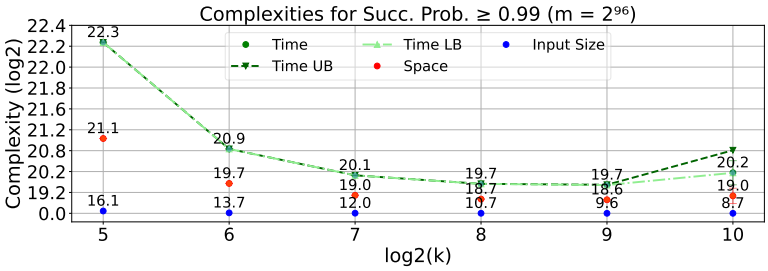
(a)



(b)



(c)



(d)

Fig. 8: Measurements of kTree's complexities for different k under fixed m's. Due to the limitation of computational resource, we were only able to measure the $k = 512$ and $k = 1024$ cases for $m = 2^{128}$. Therefore its plot is not included.

References

1. Abboud, A., Bringmann, K., Hermelin, D., Shabtay, D.: Seth-based lower bounds for subset sum and bicriteria path. In: Chan, T.M. (ed.) Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019. pp. 41–57. SIAM (2019), https://doi.org/10.1137/1.9781611975482.3
2. Abboud, A., Lewi, K.: Exact weight subgraphs and the k-sum conjecture. In: Fomin, F.V., Freivalds, R., Kwiatkowska, M.Z., Peleg, D. (eds.) Automata, Languages, and Programming - 40th International Colloquium, ICALP 2013, Riga, Latvia, July 8-12, 2013, Proceedings, Part I. Lecture Notes in Computer Science, vol. 7965, pp. 1–12. Springer (2013), https://doi.org/10.1007/978-3-642-39206-1_1
3. Abboud, A., Williams, V.V.: Popular conjectures imply strong lower bounds for dynamic problems. In: 2014 IEEE 55th Annual Symposium on Foundations of Computer Science. pp. 434–443 (2014)
4. Agrawal, S., Saha, S., Schwartzbach, N.I., Vanukuri, A., Vasudevan, P.N.: k-sum in the sparse regime: Complexity and applications. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14921, pp. 315–351. Springer (2024), https://doi.org/10.1007/978-3-031-68379-4_10
5. Ailon, N., Chazelle, B.: Lower bounds for linear degeneracy testing. J. ACM **52**(2), 157–171 (2005), https://doi.org/10.1145/1059513.1059515
6. Baran, I., Demaine, E.D., Pătraşcu, M.: Subquadratic algorithms for 3sum. Algorithmica **50**(4), 584–596 (Apr 2008), https://doi.org/10.1007/s00453-007-9036-3
7. Barequet, G., Har-Peled, S.: Polygon containment and translational min-hausdorff-distance between segment sets are 3sum-hard. Int. J. Comput. Geometry Appl. **11**, 465–474 (08 2001)
8. Benhamouda, F., Lepoint, T., Loss, J., Orrù, M., Raykova, M.: On the (in)security of ROS. J. Cryptol. **35**(4), 25 (2022), https://doi.org/10.1007/s00145-022-09436-0
9. Biryukov, A., Khovratovich, D.: Equihash: Asymmetric proof-of-work based on the generalized birthday problem. Ledger **2**, 1–30 (2017), https://ledgerjournal.org/ojs/index.php/ledger/article/view/48
10. Blum, A., Kalai, A., Wasserman, H.: Noise-tolerant learning, the parity problem, and the statistical query model. J. ACM **50**(4), 506–519 (jul 2003), https://doi.org/10.1145/792538.792543
11. Brakerski, Z., Stephens-Davidowitz, N., Vaikuntanathan, V.: On the hardness of average-case k-sum. In: Wootters, M., Sanità, L. (eds.) Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2021, August 16-18, 2021, University of Washington, Seattle, Washington, USA (Virtual Conference). LIPIcs, vol. 207, pp. 29:1–29:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021), https://doi.org/10.4230/LIPIcs.APPROX/RANDOM.2021.29
12. Bringmann, K.: A near-linear pseudopolynomial time algorithm for subset sum. In: Klein, P.N. (ed.) Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2017, Barcelona, Spain, Hotel Porta Fira, January 16-19. pp. 1073–1084. SIAM (2017), https://doi.org/10.1137/1.9781611974782.69

13. Brzuska, C., Couteau, G.: On building fine-grained one-way functions from strong average-case hardness. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13276, pp. 584–613. Springer (2022), https://doi.org/10.1007/978-3-031-07085-3_20
14. Carmosino, M.L., Gao, J., Impagliazzo, R., Mihajlin, I., Paturi, R., Schneider, S.: Nondeterministic extensions of the strong exponential time hypothesis and consequences for non-reducibility. In: Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science. p. 261–270. ITCS '16, Association for Computing Machinery, New York, NY, USA (2016), https://doi.org/10.1145/2840728.2840746
15. Chan, T.M.: More logarithmic-factor speedups for 3sum, (median, +)-convolution, and some geometric 3sum-hard problems. ACM Trans. Algorithms **16**(1), 7:1–7:23 (2020), https://doi.org/10.1145/3363541
16. Chung, E., Larsen, K.G.: Stronger 3sum-indexing lower bounds. In: Bansal, N., Nagarajan, V. (eds.) Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023. pp. 444–455. SIAM (2023), https://doi.org/10.1137/1.9781611977554.ch19
17. Coron, J., Joux, A.: Cryptanalysis of a provably secure cryptographic hash function. IACR Cryptol. ePrint Arch. p. 13 (2004), http://eprint.iacr.org/2004/013
18. Dietzfelbinger, M., Schlag, P., Walzer, S.: A subquadratic algorithm for 3xor. In: Potapov, I., Spirakis, P.G., Worrell, J. (eds.) 43rd International Symposium on Mathematical Foundations of Computer Science, MFCS 2018, August 27-31, 2018, Liverpool, UK. LIPIcs, vol. 117, pp. 59:1–59:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2018), https://doi.org/10.4230/LIPIcs.MFCS.2018.59
19. Dinur, I.: An algorithmic framework for the generalized birthday problem. Des. Codes Cryptogr. **87**(8), 1897–1926 (2019), https://doi.org/10.1007/s10623-018-00594-6
20. Dinur, I., Keller, N., Klein, O.: Fine-grained cryptanalysis: Tight conditional bounds for dense k-sum and k-xor. In: 62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022. pp. 80–91. IEEE (2021), https://doi.org/10.1109/FOCS52979.2021.00017
21. Drijvers, M., Edalatnejad, K., Ford, B., Kiltz, E., Loss, J., Neven, G., Stepanovs, I.: On the security of two-round multi-signatures. In: 2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019. pp. 1084–1101. IEEE (2019), https://doi.org/10.1109/SP.2019.00050
22. Erickson, J.: Lower bounds for linear satisfiability problems. In: Clarkson, K.L. (ed.) Proceedings of the Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, 22-24 January 1995. San Francisco, California, USA. pp. 388–395. ACM/SIAM (1995), http://dl.acm.org/citation.cfm?id=313651.313772
23. Fuchsbauer, G., Plouviez, A., Seurin, Y.: Blind schnorr signatures and signed elgamal encryption in the algebraic group model. In: Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II 30. pp. 63–95. Springer (2020)
24. Gajentaan, A., Overmars, M.H.: On a class of o(n2) problems in computational geometry. Computational Geometry **5**(3), 165–185 (1995), https://www.sciencedirect.com/science/article/pii/0925772195000222

25. Gold, O., Sharir, M.: Improved Bounds for 3SUM, k-SUM, and Linear Degeneracy. In: Pruhs, K., Sohler, C. (eds.) 25th Annual European Symposium on Algorithms (ESA 2017). Leibniz International Proceedings in Informatics (LIPIcs), vol. 87, pp. 42:1–42:13. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany (2017), http://drops.dagstuhl.de/opus/volltexte/2017/7836
26. Golovnev, A., Guo, S., Horel, T., Park, S., Vaikuntanathan, V.: Data structures meet cryptography: 3sum with preprocessing. In: Makarychev, K., Makarychev, Y., Tulsiani, M., Kamath, G., Chuzhoy, J. (eds.) Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020. pp. 294–307. ACM (2020), https://doi.org/10.1145/3357713.3384342
27. Grassi, L., Naya-Plasencia, M., Schrottenloher, A.: Quantum algorithms for the k -xor problem. In: Peyrin, T., Galbraith, S.D. (eds.) Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part I. Lecture Notes in Computer Science, vol. 11272, pp. 527–559. Springer (2018), https://doi.org/10.1007/978-3-030-03326-2_18
28. Grønlund, A., Pettie, S.: Threesomes, degenerates, and love triangles. J. ACM **65**(4) (apr 2018), https://doi.org/10.1145/3185378
29. Horowitz, E., Sahni, S.: Computing partitions with applications to the knapsack problem. J. ACM **21**(2), 277–292 (1974), https://doi.org/10.1145/321812.321823
30. Howgrave-Graham, N., Joux, A.: New generic algorithms for hard knapsacks. In: Gilbert, H. (ed.) Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings. Lecture Notes in Computer Science, vol. 6110, pp. 235–256. Springer (2010), https://doi.org/10.1007/978-3-642-13190-5_12
31. Jin, C., Wu, H.: A simple near-linear pseudopolynomial time randomized algorithm for subset sum. In: Fineman, J.T., Mitzenmacher, M. (eds.) 2nd Symposium on Simplicity in Algorithms, SOSA 2019, January 8-9, 2019, San Diego, CA, USA. OASIcs, vol. 69, pp. 17:1–17:6. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2019), https://doi.org/10.4230/OASIcs.SOSA.2019.17
32. Joux, A.: Cryptanalysis of the EMD mode of operation. In: Biham, E. (ed.) Advances in Cryptology - EUROCRYPT 2003, International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4-8, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2656, pp. 1–16. Springer (2003), https://doi.org/10.1007/3-540-39200-9_1
33. Joux, A., Kippen, H., Loss, J.: A concrete analysis of wagner's k-list algorithm over $\mathbb{F}_p$. IACR Cryptol. ePrint Arch. p. 282 (2024), https://eprint.iacr.org/2024/282
34. Kopelowitz, T., Pettie, S., Porat, E.: Higher lower bounds from the 3sum conjecture. In: Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms. p. 1272–1287. SODA '16, Society for Industrial and Applied Mathematics, USA (2016)
35. Kopelowitz, T., Porat, E.: The strong 3sum-indexing conjecture is false. CoRR **abs/1907.11206** (2019), http://arxiv.org/abs/1907.11206
36. Leurent, G., Sibleyras, F.: Low-memory attacks against two-round even-mansour using the 3-xor problem. In: Boldyreva, A., Micciancio, D. (eds.) Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference,

Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II. Lecture Notes in Computer Science, vol. 11693, pp. 210–235. Springer (2019), https://doi.org/10.1007/978-3-030-26951-7_8
37. Levieil, É., Fouque, P.: An improved LPN algorithm. In: Prisco, R.D., Yung, M. (eds.) Security and Cryptography for Networks, 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006, Proceedings. Lecture Notes in Computer Science, vol. 4116, pp. 348–359. Springer (2006), https://doi.org/10.1007/11832072_24
38. Lin, H.: On Wagner's k-Tree Algorithm Over Integers. https://github.com/haoxingl/kTreeInt (2024), https://github.com/haoxingl/kTreeInt.git
39. Lyubashevsky, V.: The parity problem in the presence of noise, decoding random linear codes, and the subset sum problem. In: Chekuri, C., Jansen, K., Rolim, J.D.P., Trevisan, L. (eds.) Approximation, Randomization and Combinatorial Optimization, Algorithms and Techniques, 8th International Workshop on Approximation Algorithms for Combinatorial Optimization Problems, APPROX 2005 and 9th InternationalWorkshop on Randomization and Computation, RANDOM 2005, Berkeley, CA, USA, August 22-24, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3624, pp. 378–389. Springer (2005), https://doi.org/10.1007/11538462_32
40. Minder, L., Sinclair, A.: The extended k-tree algorithm. J. Cryptol. **25**(2), 349–382 (2012), https://doi.org/10.1007/s00145-011-9097-y
41. Nikolic, I., Sasaki, Y.: Refinements of the k-tree algorithm for the generalized birthday problem. In: Iwata, T., Cheon, J.H. (eds.) Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29 - December 3, 2015, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9453, pp. 683–703. Springer (2015), https://doi.org/10.1007/978-3-662-48800-3_28
42. Patrascu, M.: Towards polynomial lower bounds for dynamic problems. In: Proceedings of the Forty-Second ACM Symposium on Theory of Computing. p. 603–610. STOC '10, Association for Computing Machinery, New York, NY, USA (2010), https://doi.org/10.1145/1806689.1806772
43. Patrascu, M., Williams, R.: On the possibility of faster SAT algorithms. In: Charikar, M. (ed.) Proceedings of the Twenty-First Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2010, Austin, Texas, USA, January 17-19, 2010. pp. 1065–1075. SIAM (2010), https://doi.org/10.1137/1.9781611973075.86
44. Pointcheval, D., Stern, J.: Security arguments for digital signatures and blind signatures. Journal of cryptology **13**, 361–396 (2000)
45. Puatracscu, M., Thorup, M.: The power of simple tabulation hashing. J. ACM **59**(3), 14:1–14:50 (2012), https://doi.org/10.1145/2220357.2220361
46. Schnorr, C.: Security of blind discrete log signatures against interactive attacks. In: Qing, S., Okamoto, T., Zhou, J. (eds.) Information and Communications Security, Third International Conference, ICICS 2001, Xian, China, November 13-16, 2001. Lecture Notes in Computer Science, vol. 2229, pp. 1–12. Springer (2001), https://doi.org/10.1007/3-540-45600-7_1
47. Shallue, A.: An improved multi-set algorithm for the dense subset sum problem. In: van der Poorten, A.J., Stein, A. (eds.) Algorithmic Number Theory, 8th International Symposium, ANTS-VIII, Banff, Canada, May 17-22, 2008, Proceedings. Lecture Notes in Computer Science, vol. 5011, pp. 416–429. Springer (2008), https://doi.org/10.1007/978-3-540-79456-1_28

48. Soss, M., Erickson, J., Overmars, M.: Preprocessing chains for fast dihedral rotations is hard or even impossible. Computational Geometry **26**(3), 235–246 (2003), https://www.sciencedirect.com/science/article/pii/S0925772102001566
49. Wagner, D.A.: A generalized birthday problem. In: Yung, M. (ed.) Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings. Lecture Notes in Computer Science, vol. 2442, pp. 288–303. Springer (2002), https://doi.org/10.1007/3-540-45708-9_19